implement C function, which will take WSG-84 longitude & latitude of starting location, heading to the point and distance in meters to the point from starting location and then will calculate longitude & latitude of this point.

Yes — for WGS-84 you generally want the **direct geodesic** solution: given start latitude/longitude, initial bearing, and distance, compute the destination point on the ellipsoid. A simple spherical formula exists, but it can introduce errors up to about 0.3% in typical cases; Vincenty's ellipsoidal direct method is much more appropriate for WGS-84 coordinates.[1] [2]

Recommended C function

Use this if you want good accuracy on the WGS-84 ellipsoid. Vincenty's direct method uses the WGS-84 semi-major axis $a = 6378137.0$ m and flattening $f = 1/298.257223563$.[2]

```
#include <math.h>
#include <stdbool.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

static double deg2rad(double deg) {
    return deg * M_PI / 180.0;
}

static double rad2deg(double rad) {
    return rad * 180.0 / M_PI;
}

static double normalize_lon_deg(double lon_deg) {
    while (lon_deg > 180.0) lon_deg -= 360.0;
    while (lon_deg < -180.0) lon_deg += 360.0;
    return lon_deg;
}

/*
 * Computes destination point from:
 *   lat1_deg   start latitude in degrees
 *   lon1_deg   start longitude in degrees
 *   bearing_deg initial bearing in degrees clockwise from true north
```

```
 *   distance_m distance in meters
 *
 * Outputs:
 *   *lat2_deg  destination latitude in degrees
 *   *lon2_deg  destination longitude in degrees
 *
 * Returns:
 *   true on success
 */
bool wgs84_destination_point(double lat1_deg, double lon1_deg,
                             double bearing_deg, double distance_m,
                             double *lat2_deg, double *lon2_deg)
{
    if (!lat2_deg || !lon2_deg) return false;

    const double a = 6378137.0;
    const double f = 1.0 / 298.257223563;
    const double b = a * (1.0 - f);

    const double phi1 = deg2rad(lat1_deg);
    const double lambda1 = deg2rad(lon1_deg);
    const double alpha1 = deg2rad(bearing_deg);

    const double sinAlpha1 = sin(alpha1);
    const double cosAlpha1 = cos(alpha1);

    const double tanU1 = (1.0 - f) * tan(phi1);
    const double cosU1 = 1.0 / sqrt(1.0 + tanU1 * tanU1);
    const double sinU1 = tanU1 * cosU1;

    const double sigma1 = atan2(tanU1, cosAlpha1);
    const double sinAlpha = cosU1 * sinAlpha1;
    const double cosSqAlpha = 1.0 - sinAlpha * sinAlpha;

    const double uSq = cosSqAlpha * (a * a - b * b) / (b * b);
    const double A = 1.0 + uSq / 16384.0 *
                    (4096.0 + uSq * (-768.0 + uSq * (320.0 - 175.0 * uSq)));
    const double B = uSq / 1024.0 *
                    (256.0 + uSq * (-128.0 + uSq * (74.0 - 47.0 * uSq)));

    double sigma = distance_m / (b * A);
    double sigma_prev;
    double sinSigma, cosSigma, cos2SigmaM, deltaSigma;

    int iter = 0;
    do {
        cos2SigmaM = cos(2.0 * sigma1 + sigma);
        sinSigma = sin(sigma);
        cosSigma = cos(sigma);

        deltaSigma = B * sinSigma *
            (cos2SigmaM + B / 4.0 *
            (cosSigma * (-1.0 + 2.0 * cos2SigmaM * cos2SigmaM) -
             B / 6.0 * cos2SigmaM *
             (-3.0 + 4.0 * sinSigma * sinSigma) *
             (-3.0 + 4.0 * cos2SigmaM * cos2SigmaM)));
```

```
        sigma_prev = sigma;
        sigma = distance_m / (b * A) + deltaSigma;
    } while (fabs(sigma - sigma_prev) > 1e-12 && ++iter < 200);

    if (iter >= 200) return false;

    const double tmp = sinU1 * sinSigma - cosU1 * cosSigma * cosAlpha1;

    const double phi2 = atan2(
        sinU1 * cosSigma + cosU1 * sinSigma * cosAlpha1,
        (1.0 - f) * sqrt(sinAlpha * sinAlpha + tmp * tmp)
    );

    const double lambda = atan2(
        sinSigma * sinAlpha1,
        cosU1 * cosSigma - sinU1 * sinSigma * cosAlpha1
    );

    const double C = f / 16.0 * cosSqAlpha * (4.0 + f * (4.0 - 3.0 * cosSqAlpha));

    const double L = lambda - (1.0 - C) * f * sinAlpha *
        (sigma + C * sinSigma *
        (cos2SigmaM + C * cosSigma *
        (-1.0 + 2.0 * cos2SigmaM * cos2SigmaM)));

    const double lambda2 = lambda1 + L;

    *lat2_deg = rad2deg(phi2);
    *lon2_deg = normalize_lon_deg(rad2deg(lambda2));

    return true;
}
```

How to use it

The input bearing should be in degrees clockwise from **true north**, with 0° = north, 90° = east, and so on. Angles must be converted to radians internally for trig functions, and longitude is typically normalized back to the $[-180, 180]$ range after the calculation.[1] [2]

```
#include <stdio.h>

int main(void) {
    double lat2, lon2;

    if (wgs84_destination_point(52.2297, 21.0122, 45.0, 10000.0, &lat2, &lon2)) {
        printf("Destination: lat=%.8f lon=%.8f\n", lat2, lon2);
    } else {
        printf("Calculation failed\n");
    }

    return 0;
}
```

If you want simpler code

A shorter spherical formula is available for "destination point given distance and bearing," using mean Earth radius $R \approx 6371000$ m, but that model ignores ellipsoidal effects and can be off by up to about 0.3% in many situations. If your distances are small and meter-level precision is not critical, that version may still be acceptable.[1]

```
void destination_point_sphere(double lat1_deg, double lon1_deg,
                              double bearing_deg, double distance_m,
                              double *lat2_deg, double *lon2_deg)
{
    const double R = 6371000.0;

    double phi1 = deg2rad(lat1_deg);
    double lambda1 = deg2rad(lon1_deg);
    double theta = deg2rad(bearing_deg);
    double delta = distance_m / R;

    double phi2 = asin(sin(phi1) * cos(delta) +
                       cos(phi1) * sin(delta) * cos(theta));

    double lambda2 = lambda1 + atan2(sin(theta) * sin(delta) * cos(phi1),
                                     cos(delta) - sin(phi1) * sin(phi2));

    *lat2_deg = rad2deg(phi2);
    *lon2_deg = normalize_lon_deg(rad2deg(lambda2));
}
```

Practical note

If you need the most robust modern solution, Karney/GeographicLib is generally preferred over Vincenty because it is designed to converge reliably even in difficult cases where Vincenty can fail, while Vincenty is still widely used and accurate for normal cases.[2]

If you want, I can also provide:

- a **header** + **source** version,
- a **unit test** with known WGS-84 examples,
- or a version using **GeographicLib in C/C++**.

⁂

add more code comments to wgs84_destination_point describing what each local variable means.

Sure — below is the same `wgs84_destination_point()` function with much heavier inline documentation for each local variable. The variables follow Vincenty's direct method on the WGS-84 ellipsoid, where the key intermediate quantities include the reduced latitude U_1, the spherical arc length σ, the azimuth-related term α, and correction terms such as u^2, A, B, and $\Delta\sigma$.[11] [12] [13]

Commented function

```
#include <math.h>
#include <stdbool.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

static double deg2rad(double deg) {
    return deg * M_PI / 180.0;
}

static double rad2deg(double rad) {
    return rad * 180.0 / M_PI;
}

static double normalize_lon_deg(double lon_deg) {
    while (lon_deg > 180.0) lon_deg -= 360.0;
    while (lon_deg < -180.0) lon_deg += 360.0;
    return lon_deg;
}

/*
 * Vincenty direct solution on the WGS-84 ellipsoid.
 *
 * Inputs:
 *   lat1_deg    - start latitude, degrees
 *   lon1_deg    - start longitude, degrees
 *   bearing_deg - initial azimuth / heading, degrees clockwise from true north
 *   distance_m  - distance to travel along the geodesic, meters
 *
 * Outputs:
 *   lat2_deg    - destination latitude, degrees
 *   lon2_deg    - destination longitude, degrees
 *
 * Returns:
 *   true  - converged successfully
 *   false - invalid output pointers or iteration did not converge
 */
bool wgs84_destination_point(double lat1_deg, double lon1_deg,
                             double bearing_deg, double distance_m,
```

```
                              double *lat2_deg, double *lon2_deg)
{
    if (!lat2_deg || !lon2_deg) return false;

    /* WGS-84 ellipsoid constants:
     * a = semi-major axis (equatorial radius)
     * f = flattening
     * b = semi-minor axis (polar radius)
     */
    const double a = 6378137.0;
    const double f = 1.0 / 298.257223563;
    const double b = a * (1.0 - f);

    /* Convert input geographic coordinates and heading to radians
     * because C trig functions use radians.
     */
    const double phi1 = deg2rad(lat1_deg);     /* geodetic latitude of start point */
    const double lambda1 = deg2rad(lon1_deg);  /* longitude of start point */
    const double alpha1 = deg2rad(bearing_deg);/* initial forward azimuth at start *

    /* Trig values of the initial azimuth.
     * Precomputed once because they are reused several times.
     */
    const double sinAlpha1 = sin(alpha1);      /* sin(initial bearing) */
    const double cosAlpha1 = cos(alpha1);      /* cos(initial bearing) */

    /* Reduced latitude U1.
     * Vincenty works on the auxiliary sphere rather than directly on the ellipsoid.
     * U1 is the "parametric" or "reduced" latitude corresponding to phi1.
     *
     * tanU1 = (1-f) * tan(phi1)
     */
    const double tanU1 = (1.0 - f) * tan(phi1);   /* tangent of reduced latitude at st
    const double cosU1 = 1.0 / sqrt(1.0 + tanU1 * tanU1); /* cosine of reduced latitu
    const double sinU1 = tanU1 * cosU1;           /* sine of reduced latitude U1 */

    /* sigma1 is the angular distance on the auxiliary sphere
     * from the equator to the start point, measured along the geodesic.
     */
    const double sigma1 = atan2(tanU1, cosAlpha1);

    /* sinAlpha is the azimuth of the geodesic at the equator
     * on the auxiliary sphere; it stays constant along the geodesic.
     */
    const double sinAlpha = cosU1 * sinAlpha1;

    /* cosSqAlpha = cos(alpha)^2
     * This appears repeatedly in Vincenty's series expansions.
     */
    const double cosSqAlpha = 1.0 - sinAlpha * sinAlpha;

    /* uSq is the "ellipsoid shape" parameter for this geodesic.
     * It depends on Earth flattening and on the azimuth of the path.
     * Larger uSq means a stronger ellipsoidal correction is needed.
     */
    const double uSq = cosSqAlpha * (a * a - b * b) / (b * b);
```

```
/* A and B are series coefficients used to correct the spherical
 * arc-length solution so it matches the ellipsoidal geodesic.
 */
const double A = 1.0 + uSq / 16384.0 *
                (4096.0 + uSq * (-768.0 + uSq * (320.0 - 175.0 * uSq)));
const double B = uSq / 1024.0 *
                (256.0 + uSq * (-128.0 + uSq * (74.0 - 47.0 * uSq)));

/* sigma is the current estimate of the angular distance on the
 * auxiliary sphere from the start point to the destination point.
 *
 * First guess: distance / (b*A)
 */
double sigma = distance_m / (b * A);

/* sigma_prev stores the previous iteration's sigma so we can test convergence.
double sigma_prev;

/* These are updated each iteration:
 * sinSigma    = sin(sigma)
 * cosSigma    = cos(sigma)
 * cos2SigmaM  = cos(2*sigma_m), where sigma_m is the angular distance
 *               from the equator to the midpoint of the geodesic segment
 *               on the auxiliary sphere
 * deltaSigma  = correction term applied to sigma
 */
double sinSigma, cosSigma, cos2SigmaM, deltaSigma;

/* Iteration counter to avoid infinite loops if convergence fails. */
int iter = 0;

do {
    /* cos(2*sigma_m), where:
     * 2*sigma_m = 2*sigma1 + sigma
     *
     * This midpoint-related term appears in Vincenty's correction formula.
     */
    cos2SigmaM = cos(2.0 * sigma1 + sigma);

    /* Trig values of current sigma estimate. */
    sinSigma = sin(sigma);
    cosSigma = cos(sigma);

    /* Ellipsoidal correction to sigma.
     * This is the key term that refines the auxiliary-sphere arc length
     * into the ellipsoidal geodesic solution.
     */
    deltaSigma = B * sinSigma *
        (cos2SigmaM + B / 4.0 *
        (cosSigma * (-1.0 + 2.0 * cos2SigmaM * cos2SigmaM) -
         B / 6.0 * cos2SigmaM *
         (-3.0 + 4.0 * sinSigma * sinSigma) *
         (-3.0 + 4.0 * cos2SigmaM * cos2SigmaM)));

    /* Save previous sigma before updating. */
```

```
        sigma_prev = sigma;

        /* New refined sigma estimate. */
        sigma = distance_m / (b * A) + deltaSigma;

    } while (fabs(sigma - sigma_prev) > 1e-12 && ++iter < 200);

    if (iter >= 200) return false;

    /* tmp is an intermediate quantity used in the final latitude formula
     * and in the reverse-azimuth expression.
     */
    const double tmp = sinU1 * sinSigma - cosU1 * cosSigma * cosAlpha1;

    /* phi2 is the destination geodetic latitude on the ellipsoid. */
    const double phi2 = atan2(
        sinU1 * cosSigma + cosU1 * sinSigma * cosAlpha1,
        (1.0 - f) * sqrt(sinAlpha * sinAlpha + tmp * tmp)
    );

    /* lambda is the difference in longitude on the auxiliary sphere
     * before applying the ellipsoidal longitude correction.
     */
    const double lambda = atan2(
        sinSigma * sinAlpha1,
        cosU1 * cosSigma - sinU1 * sinSigma * cosAlpha1
    );

    /* C is another series coefficient used in the longitude correction term. */
    const double C = f / 16.0 * cosSqAlpha * (4.0 + f * (4.0 - 3.0 * cosSqAlpha));

    /* L is the corrected ellipsoidal longitude difference from start to destinatior
     * This is what must be added to lambda1 to get the destination longitude.
     */
    const double L = lambda - (1.0 - C) * f * sinAlpha *
        (sigma + C * sinSigma *
        (cos2SigmaM + C * cosSigma *
        (-1.0 + 2.0 * cos2SigmaM * cos2SigmaM)));

    /* Absolute longitude of destination point in radians. */
    const double lambda2 = lambda1 + L;

    /* Convert final answers back to degrees. */
    *lat2_deg = rad2deg(phi2);
    *lon2_deg = normalize_lon_deg(rad2deg(lambda2));

    return true;
}
```

Variable guide

Here is the practical meaning of the most important locals in the function. Vincenty's direct method works by mapping the ellipsoid to an auxiliary sphere, solving the path there, then applying series corrections back to the ellipsoid.[12] [13] [11]

Variable	Meaning
`a`	WGS-84 semi-major axis, Earth equatorial radius. [13]
`f`	WGS-84 flattening, describing how much the ellipsoid is squashed from a sphere. [13]
`b`	WGS-84 semi-minor axis, Earth polar radius. [13]
`phi1`	Start geodetic latitude, in radians. [13]
`lambda1`	Start longitude, in radians. [13]
`alpha1`	Initial azimuth or heading from the start point, in radians. [13] [12]
`U1` via `tanU1/sinU1/cosU1`	Reduced latitude of the start point on the auxiliary sphere. [13] [11]
`sigma1`	Arc from the equator to the start point on the auxiliary sphere, measured along the geodesic. [11] [12]
`sinAlpha`	Constant azimuth-related term of the geodesic on the auxiliary sphere. [11] [12]
`cosSqAlpha`	$\cos^2 \alpha$, used in the correction series. [11] [12]
`uSq`	Ellipsoid-dependent parameter controlling the strength of correction terms. [13] [12]
`A`, `B`	Series-expansion coefficients used to refine the spherical solution into an ellipsoidal one. [13] [12]
`sigma`	Current estimate of angular distance from start to destination on the auxiliary sphere. [13] [12]
`cos2SigmaM`	Cosine of twice the midpoint arc angle on the auxiliary sphere. [11] [12]
`deltaSigma`	Correction applied to `sigma` during iteration. [11] [12]
`tmp`	Intermediate quantity used in the final latitude and azimuth-related expressions. [12]
`lambda`	Auxiliary-sphere longitude difference before final ellipsoidal correction. [12]
`C`	Coefficient used in the longitude correction. [12]
`L`	Corrected geodesic longitude difference on the ellipsoid. [12]
`lambda2`	Final destination longitude in radians. [12]

Small suggestion

If you want the code even easier to maintain, I can rewrite it one step further by:

- renaming variables like `phi1`, `lambda1`, `sigma1`, `uSq` into longer descriptive names, or
- splitting the algorithm into small helper blocks with comments like “prepare constants,” “iterate sigma,” and “compute final lat/lon.”

⁂

1. https://www.movable-type.co.uk/scripts/latlong.html
2. http://docs.ros.org/en/melodic/api/gtsam/html/examples_2example-Geodesic_8cpp_source.html
3. https://www.ancientportsantiques.com/wp-content/uploads/Documents/ETUDESarchivees/MedNavigationRoutes/MedNav/TrigoSpherique.pdf
4. https://www.movable-type.co.uk/scripts/latlong-vincenty.html
5. http://www.movable-type.co.uk/scripts/latlong.html?from=49.243824%2C-121.887340&to=49.227648%2C-121.89631
6. https://www.rdocumentation.org/packages/geographiclib/versions/0.4.1/topics/geodesic_direct
7. https://stackoverflow.com/questions/10119479/calculating-lat-and-long-from-bearing-and-distance
8. https://blog.naver.com/naval88/80112683397
9. https://github.com/mitre/geodetic_library
10. https://www.movable-type.co.uk/scripts/latlong-vectors.html
11. https://handwiki.org/wiki/Earth:Vincenty's_formulae
12. https://lists.osgeo.org/pipermail/openlayers-dev/2007-October/001543.html
13. http://docs.ros.org/en/melodic/api/gtsam/html/examples_2example-Geodesic_8cpp_source.html
14. https://www.boost.org/doc/libs/1_71_0/boost/geometry/formulas/vincenty_direct.hpp
15. https://search.r-project.org/CRAN/refmans/geosphere/html/distVincentyEllipsoid.html
16. https://phys.libretexts.org/Courses/Prince_Georges_Community_College/General_Physics_I:_Classical_Mechanics/56:_Geodesy/56.05:_Vincentys_Formul-_Direct_Problem
17. https://github.com/Turfjs/turf-vincenty-direct
18. https://en.wikipedia.org/wiki/Vincenty's_formulae
19. https://www.movable-type.co.uk/scripts/geodesy/docs/module-latlon-ellipsoidal-vincenty-LatLonEllipsoidal_Vincenty.html
20. https://www.boost.org/doc/libs/1_66_0/boost/geometry/formulas/vincenty_direct.hpp
21. http://fmwww.bc.edu/repec/bocode/v/vincenty.html